

# Vector Databases & RAG Cheat Sheet

Baseline: Python 3.12, sentence-transformers 6.1, FAISS 1.15.1, sqlite-vec 0.1.9, pgvector 0.8.6, Ollama 0.34, LangChain 1.4 / langchain-core 1.6 / LangGraph 1.2, LangChain4j 1.20.1, Spring Boot 4.1 / Spring AI 2.0.1, RAGAS 0.4.3. Quick reference only — verify against the official docs before production use.

| ENGINE COMPARISON MATRIX — AS OF 2026-09-26 |                                     |                                      |                                                                                         |                                     |                                  |              |                                    |  |
|---------------------------------------------|-------------------------------------|--------------------------------------|-----------------------------------------------------------------------------------------|-------------------------------------|----------------------------------|--------------|------------------------------------|--|
| Engine                                      | Best for                            | Scaling                              | Index location                                                                          | Pricing model                       | AWS                              | Google Cloud | Azure                              |  |
| pgvector                                    | Already on Postgres                 | Vertical; Citus/app sharding         | RAM (HNSW/IVFlat)                                                                       | Underlying Postgres compute+storage | 1P                               | 1P           | 1P                                 |  |
| sqlite-vec                                  | Embedded/edge/offline               | Single writer, small scale           | SQLite file; brute-force KNN, no ANN index                                              | Free (embedded)                     | Self                             | Self         | Self                               |  |
| Qdrant                                      | Self-hosted purpose-built, filters  | Sharding+replication                 | RAM or on-disk/mmap                                                                     | Resource (vCPU/RAM/disk)            | V                                | V            | V                                  |  |
| Milvus                                      | Billion-scale distributed           | Disaggregated storage/compute        | RAM, DiskANN, GPU CAGRA                                                                 | Serverless vCU or CU-hours          | V                                | V            | V                                  |  |
| Weaviate                                    | Built-in vectorizer+hybrid          | Multi-tenant sharding                | RAM HNSW (compressible)                                                                 | Plan-based (Free/Flex/Premium)      | V                                | V (Flex+)    | V (Premium)                        |  |
| Pinecone                                    | Zero-ops serverless                 | Storage/compute separated            | Object storage, serverless                                                              | Read/write units + storage          | V                                | V            | V                                  |  |
| Redis                                       | Lowest latency, cache/agent memory  | Cluster, in-memory                   | RAM (FLAT/HNSW/SVS)                                                                     | Memory size (+throughput)           | 1P                               | 1P           | 1P                                 |  |
| Elasticsearch                               | Lexical+vector hybrid, aggregations | Distributed shards                   | JVM heap + off-heap HNSW                                                                | Cloud Hosted/Serverless VCU         | V                                | V            | V                                  |  |
| MongoDB Atlas                               | Vectors next to operational docs    | Dedicated Search Nodes               | Separate mongot index                                                                   | Cluster tier + Search Nodes         | V: Atlas (+1P-compat DocumentDB) | V: Atlas     | V: Atlas (+1P-compat Cosmos vCore) |  |
| Neo4j                                       | GraphRAG, graph+vector              | Limited write scaling                | RAM, Lucene HNSW                                                                        | Instance size per hour              | V (AuraDB)                       | V (AuraDB)   | V (AuraDB)                         |  |
| Couchbase                                   | KV+SQL++ +search+vector in one      | Cluster; Server 8.0 Hyperscale Index | Disk-based Hyperscale (DiskANN/Vamana-inspired + IVF), Composite GSI, or Search-service | Compute+storage (Capella)           | V (Capella)                      | V (Capella)  | V (Capella)                        |  |

VECTOR DB CATEGORIES

- Libraries:** FAISS — building blocks only, no server/persistence.
- SQL extensions:** pgvector, sqlite-vec — vectors alongside existing tables.
- Purpose-built:** Qdrant, Milvus, Weaviate, Pinecone, Chroma, LanceDB.
- Search engines:** Elasticsearch, OpenSearch, Solr/Lucene — lexical+vector hybrid.
- Multi-model/NoSQL:** MongoDB, Redis, Couchbase, Neo4j — vectors next to other data models.

QUANTIZATION

- Scalar (SQ / int8):** per-dimension linear quantization, ~4x smaller, small recall hit.
- Product (PQ):** split vector into subvectors, quantize each with a codebook — large compression, lower recall.
- Binary:** 1 bit/dim, ~32x smaller & fastest — use with a **rescoring** pass over full-precision vectors on the shortlist to recover recall.

RAG PIPELINE

**Ingest:** load → chunk → embed → store (vectors + metadata).

**Query:** embed query → retrieve top-k → (filter/re-rank) → augment prompt → generate → (cite sources).

MEMORY & SEMANTIC CACHE

**Agent memory:** store past turns/facts as embeddings, retrieve relevant memory into the prompt.

**Semantic cache:** on a new query, do a similarity search over past query→answer pairs; serve a cached answer above a similarity threshold instead of re-calling the LLM. Redis is the common low-latency backend for both.

EMBEDDINGS

- Pick a model for domain fit, language, context length and licence — not just MTEB rank.
- Normalize** vectors (unit length) when using cosine/dot so both agree.
- Use the **same model** to embed the corpus and the query — never mix models/versions.
- Dimensions:** higher = more nuance & cost.
- Matryoshka** models let you truncate the vector (e.g. 768→256) and keep most retrieval quality.
- Re-embed the whole corpus whenever the model changes.

FILTERING

- Pre-filter:** filter first, then brute-force the survivors — exact but slow at scale.
- Post-filter:** ANN search then discard non-matches — fast but can starve results if selective.
- Iterative** (e.g. pgvector hnsw, iterative\_scan): expand the ANN scan until enough filtered matches are found.
- Overfetch:** request k\*N candidates, then filter/re-rank down to k as a cheap approximation.

CHUNKING RULES OF THUMB

- Chunk by semantic unit (heading/paragraph) before falling back to fixed size.
- Typical size: a few hundred tokens; add 10-20% overlap to preserve context across boundaries.
- Keep chunk metadata (source, section, page) alongside the vector for citation and filtering.
- Re-chunk and re-embed when the splitting strategy or model changes.

LANGCHAIN / LANGCHAIN4J

```
store = PGVector(embeddings=..., ...)
retriever = store.as_retriever()
splitter = RecursiveCharacterTextSplitter(...)
agent = create_agent(model, tools=[retriever_tool])
# LangGraph for multi-step agent control flow
```

**LangChain4j:** EmbeddingStore, EmbeddingStoreIngestor, AIServices one-liners for a RAG-backed interface.

SIMILARITY METRICS

| Metric       | Formula                     | pgvector   | sqlite-vec   | FAISS    |
|--------------|-----------------------------|------------|--------------|----------|
| L2           | $\sqrt{\sum (a_i - b_i)^2}$ | <->        | l2 (default) | FlatL2   |
| IP           | $\sum a_i b_i$              | <#> (neg.) | n/a          | FlatIP   |
| Cosine dist. | $1 - a \cdot b / ( a  b )$  | <->        | cosine       | norm.+IP |
| L1           | $\sum  a_i - b_i $          | <+>        | l1           | n/a      |

sqlite-vec values go in the vec0 column's distance\_metric= option (no inner product); FAISS = Index+ class.

Cosine: most embeddings. Dot: pre-normalized vectors, max throughput. Euclidean: raw magnitude matters. Manhattan: sparse/outlier-robust.

HYBRID SEARCH & RE-RANKING

- BM25**/full-text lexical score fused with vector similarity.
- Weighted fusion:** normalize both score lists (min-max) then combine linearly — scale-sensitive.
- RRF** (Reciprocal Rank Fusion): rank-based, no normalization needed.

$$RRF(d) = \sum 1 / (k + \text{rank}_r(d)), k=60$$

**Re-rank** the fused shortlist with a cross-encoder for precision. **MMR** trades relevance for diversity when selecting the final context set.

RETRIEVAL STRATEGIES

- Multi-query:** LLM rewrites the query several ways, union the results.
- HyDE:** embed a hypothetical answer instead of the raw query.
- Parent-document:** retrieve small chunks, return their larger parent for context.
- Multilevel:** coarse retrieval over summaries, then fine retrieval within selected docs.
- Routing:** pick which index/collection to query based on the question.

SPRING AI

Add a spring-ai-starter-vector-store-\* dependency, configure spring.ai.vectorstore.\*.

```
vectorStore.similaritySearch(
    SearchRequest.builder().query(q).topK(5).build());
```

Advisors: QuestionAnswerAdvisor, RetrievalAugmentationAdvisor, VectorStoreChatMemoryAdvisor. Expose tools with @Tool.

ANN INDEXES

- Flat:** brute-force, exact, no build — ground truth / small collections.
- IVF:** nList coarse cells; search probes nprobe of them — more nprobe = better recall, slower.
- HNSW:** M edges/node, ef\_construction build-time candidates, ef\_search query-time candidates. Higher = better recall, more RAM/latency.

bytes/vector = d\*4 + M\*2\*4  
e.g. d=768, M=16 → 3072+128 = 3200 B

PGVECTOR ESSENTIALS

Types: vector, halfvec, sparsevec, bit.

```
CREATE INDEX ... ON t
USING vec0(embedding vector_cosine_ops)
WITH (m=16, ef_construction=64);
SET hnsw.ef_search = 100;
SET hnsw.iterative_scan = relaxed_order;
```

Ops: <-> L2, <#> inner product, <=> cosine, <+> L1.

PROMPT RULES

- Ask for **citations** back to the source chunk/document.
- Ordering:** put the most relevant context closest to the question (recency/position bias).
- Explicitly allow **"I don't know"** when context is insufficient.
- Low **temperature** for factual RAG answers.
- Size **num\_ctx** to fit the retrieved context plus history plus headroom for the answer.

EVALUATION METRICS

- Recall@k:** fraction of relevant docs found in the top k.
- MRR:** mean reciprocal rank of the first relevant hit.
- nDCG:** rank-position-discounted relevance, graded not just binary.
- RAGAS**-style: faithfulness, answer relevance, context precision/recall — LLM-judged end-to-end.

SQLITE-VEC ESSENTIALS

```
CREATE VIRTUAL TABLE vec_chunks
USING vec0(chunk_id integer primary key,
    embedding float[768] distance_metric=cosine);

SELECT chunk_id, distance FROM vec_chunks
WHERE embedding MATCH :q AND k = 5;
```

Pure-C SQLite extension, embedded/offline, single writer. Pair with FTS5 for hybrid search.

AGENTIC RAG LOOP

Plan → call a retrieval tool (possibly several times, reformulating) → evaluate sufficiency → call other tools as needed → synthesize a grounded answer — instead of one fixed retrieve-then-generate pass.

PRODUCTION & SECURITY CHECKLIST

- Access control / RBAC on the store; encrypt in transit and at rest.
- Monitor recall/latency drift, not just uptime; alert on index staleness.
- Version embeddings & index config; plan a re-embedding migration path.
- Backups/snapshots; capacity-plan RAM using the per-vector memory formula.
- Rate-limit and sanitize inputs; guard against prompt injection from retrieved content.